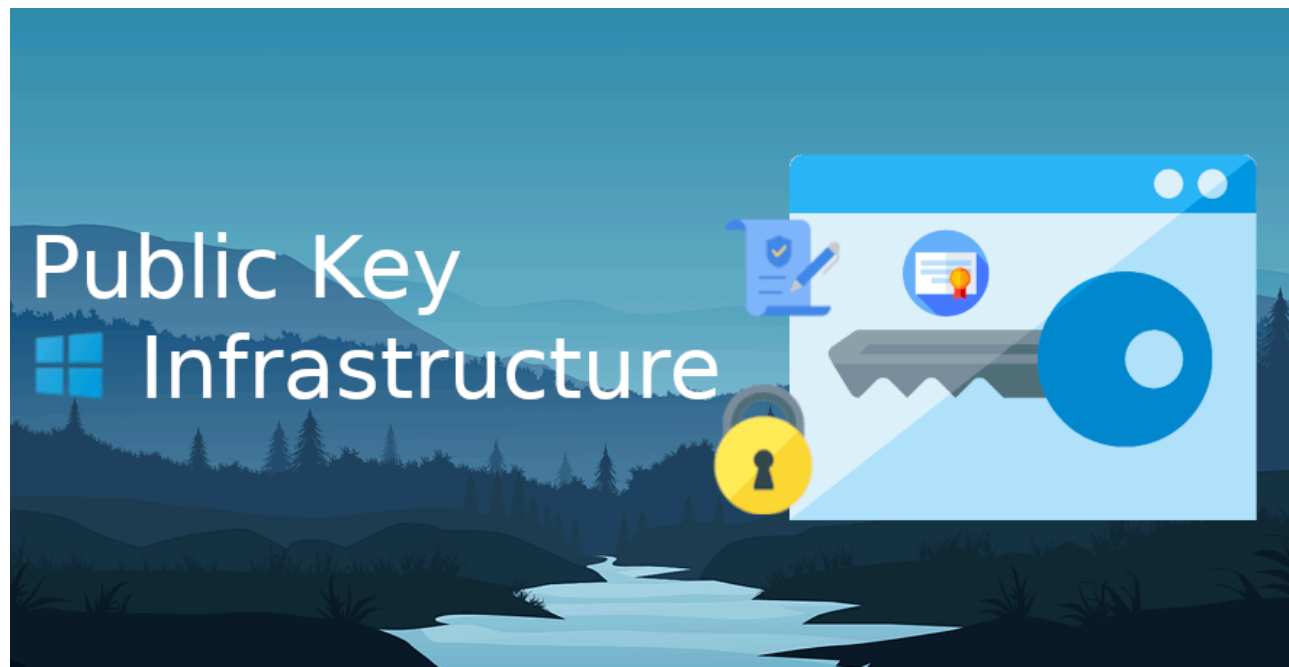


Build a PKI with PowerShell – Part 4 – Enterprise CA – Michael Waterman

 michaelwaterman.nl/2026/01/03/how-to-build-a-pki-with-powershell-part-4-enterprise-ca

Michael Waterman

January 3, 2026



Other parts in this series

[How to: Build a PKI with PowerShell – Part 1 – Preparation](#)

[How to: Build a PKI with PowerShell – Part 2 – IIS WebServer](#)

[How to: Build a PKI with PowerShell – Part 3 – Offline Root CA](#)

In the [previous parts](#) of this series, we've laid the foundation of my PKI infrastructure. I've designed the architecture, prepared the environment, built the web distribution layer, and established a secure and isolated Root Certificate Authority. With that foundation in place, I can now move on to the component that will actually issue certificates: the *Enterprise Certification Authority*.

This is the system that clients and servers interact with on a daily basis. It handles certificate enrollment, renewal, and integration with Active Directory. While the Root CA defines trust, the Enterprise CA operationalizes it.

The role of the Enterprise CA

Unlike the Root CA, the Enterprise CA is:

- Domain-joined

- Actively used by clients and services
- Integrated with Active Directory
- Designed to operate continuously

It issues certificates for:

- Users
- Devices
- Services
- Infrastructure components
- And a whole lot more!

Because of this, the Enterprise CA sits directly in the operational path of authentication and trust. It must be stable, predictable, and properly integrated into the domain environment.

What I'll do in this part

In this section, I'll walk through:

- Installing the Enterprise CA role
- Creating the certificate request
- Submitting the request to the Root CA
- Completing the CA installation
- Publishing certificates and trust information to Active Directory

This is where everything comes together, so let's go for the final steps! Unlike the Root CA, this system is online, domain-joined, and actively involved in daily operations such as certificate enrollment and renewal. Although this blog concentrates on the installation of the Enterprise CA only, make sure that your server is properly configured, hardened and in operation management before continuing.

Step 1 – Creating the `capolicy.inf` for the Enterprise CA

Required permissions

- Local Administrator on the Enterprise CA
- *Enterprise Admin* (temporary, during CA installation only)
- Delegated permissions for publishing the information into AD (A blog I'll write one day)

Why:

Installing an Enterprise CA requires writing PKI-related objects to Active Directory (Configuration partition). This is why Enterprise Admin rights are required *only during installation*.

Tip! The following steps are high secure operations, *Enterprise Admin permissions are not required for day-to-day CA operations and should be removed after installation.*

Before installing the Enterprise Certification Authority, we once again start with the `capolicy.inf` file. Just like with the Root CA, this file influences how the CA is created, but its role here is slightly different.

While the Root CA defines the trust anchor, the Enterprise CA operates *within* that trust. Its `capolicy.inf` therefore focuses less on long-term authority and more on consistency, predictability, and alignment with the Root CA configuration.

How the Enterprise CA differs from the Root CA

There are a few important distinctions to keep in mind:

- The Enterprise CA *inherits trust* from the Root CA
- Certificate lifetime is constrained by the Root CA, it can not be longer than the lifetime of the Root CA certificate.
- Some values act as defaults rather than hard limits
- The CA is integrated with Active Directory

This means the `capolicy.inf` is still important, but its influence is more subtle and forward-looking, especially for renewal scenarios.

What the Enterprise CA `capolicy.inf` is used for

During installation and future renewals, the file controls:

- Default certificate validity settings
- Cryptographic behavior (hashing, signing consistency)
- Extension behavior that must remain predictable across renewals

Not all values are applied immediately, but defining them early avoids inconsistencies later, this is pretty much the same behavior as with the Root CA.

Creating the `capolicy.inf` file

Just like with the Root CA, the file must exist **before** the CA role is installed.

Below is the structure used in this setup. It mirrors the Root CA approach but reflects the Enterprise CA's role.

```
$CAPolicy = Join-Path -Path $env:SystemRoot -ChildPath "CAPolicy.inf"
```

```
# Remove existing CAPolicy.inf (if it exists) to avoid inheriting old or unintended
```

```

settings
if (Test-Path -Path $CAPolicy) {
    Remove-Item -Path $CAPolicy -Force
}

# Create the CAPolicy.inf file (ASCII/ANSI encoding is required for proper
processing by ADCS)
"[Version]" | Out-File -Encoding ascii -FilePath $CAPolicy
Add-Content -Path $CAPolicy -Value 'Signature="$Windows NT$"'
Add-Content -Path $CAPolicy -Value ""

# CA defaults used for installation and future CA certificate renewal behavior
Add-Content -Path $CAPolicy -Value "[Certsrv_Server]"
Add-Content -Path $CAPolicy -Value "CRLPeriod=Week"
Add-Content -Path $CAPolicy -Value "CRLPeriodUnits=1"
Add-Content -Path $CAPolicy -Value "CRLDeltaPeriod=Day"
Add-Content -Path $CAPolicy -Value "CRLDeltaPeriodUnits=1"
Add-Content -Path $CAPolicy -Value "LoadDefaultTemplates=0"

# Cryptographic defaults
Add-Content -Path $CAPolicy -Value "CNGHashAlgorithm=SHA256"
Add-Content -Path $CAPolicy -Value "AlternateSignatureAlgorithm=0"
Add-Content -Path $CAPolicy -Value ""

# Certificate policy information included in the Enterprise CA certificate
Add-Content -Path $CAPolicy -Value "[PolicyStatementExtension]"
Add-Content -Path $CAPolicy -Value "Policies = CorpPolicy"
Add-Content -Path $CAPolicy -Value ""

Add-Content -Path $CAPolicy -Value "[CorpPolicy]"
Add-Content -Path $CAPolicy -Value "OID = 1.3.6.1.4.1.#####.1.1"
Add-Content -Path $CAPolicy -Value "URL=http://trust.domain.suffix/cps/cps.html"
Add-Content -Path $CAPolicy -Value ""

# CDP and AIA publication paths for certificates issued by this CA
# are configured later in the CA configuration, not hardcoded here.

# Force KeyUsage extension to be marked as Critical in the Enterprise CA
certificate
Add-Content -Path $CAPolicy -Value "[Extensions]"
Add-Content -Path $CAPolicy -Value "2.5.29.15=AwIBhg=="
Add-Content -Path $CAPolicy -Value "Critical=2.5.29.15"

```

Note: Replace the ##### with your own PEN number, see the section “Policy OID + CPS URL: make certificates self-describing” down below for more information.

Note: The “RenewalKeyLength” setting under “[Certsrv_Server]” is typically not required. By default, the CA will reuse the existing key length during renewal. This setting is only relevant when performing a CA renewal with a new key and explicitly changing the key length. In modern Windows Server versions, it is safe to omit this value unless a key size change is required.

What does this all mean?

For the Enterprise CA, `capolicy.inf` is less about establishing trust (that's the Root CA's job) and more about enforcing consistent operational behavior and clean certificate metadata from day one.

CRL and Delta CRL behavior

You're explicitly setting:

- `CRLPeriod=Week / CRLPeriodUnits=1`
- `CRLDeltaPeriod=Day / CRLDeltaPeriodUnits=1`

That makes perfect sense for an online Enterprise CA. Unlike the Offline Root CA, this CA is actively issuing certificates and you want revocation information to refresh predictably. Weekly base CRLs with daily delta CRLs is a pragmatic balance between client traffic and revocation freshness.

Don't load default templates

`LoadDefaultTemplates=0` prevents the CA from automatically enabling Microsoft's default certificate templates. This is one of those "save yourself later" settings. It forces you to enable templates intentionally, instead of inheriting whatever defaults happen to be present. Less clutter, less risk, and far fewer surprises.

Policy OID + CPS URL: make certificates self-describing

The `PolicyStatementExtension` section ties your CA to:

- a dedicated policy name (`CorpPolicy`)
- your registered/private enterprise OID (`1.3.6.1.4.1.55468.1.1`)
- a CPS URL hosted on your web server

This is what turns your certificates into something that's not just cryptographically valid, but also *operationally meaningful*. Anyone inspecting the certificate can see: "this certificate belongs to this PKI, under this policy, with documentation here." The long OID number sequence, `1.3.6.1.4.1.55468.1.1`, the 55468 part is your unique Private Enterprise Number as I discussed in part one of the series. The presiding part `1.3.6.1.4.1` is a registration for private enterprise use, as defined by IANA. If you want to know more about OID's, check out my blog on that, [Part 5 – PKI Best Practices: Creating Unique Object Identifiers \(OIDs\)](#).

CDP/AIA URLs: keep distribution predictable

By defining the HTTP URLs for:

- CRL distribution (CDP)
- CA certificate retrieval (AIA)

You ensure certificates consistently point to the web distribution layer you built in Part 2. That keeps your CA independent from internal paths and avoids certificates being stamped with awkward or unreachable default locations.

KeyUsage marked as Critical

Finally, you force the KeyUsage extension to be *Critical*:

- 2.5.29.15=AwIBhg==
- Critical=2.5.29.15

This makes sure relying parties interpret the certificate's intended use strictly. It's a small line in a config file, but it's one of those details that keeps your PKI output clean and standards-aligned.

"Just like with the Root CA: if this file isn't present before CA installation, Windows will happily ignore it."

Step 2 – Installing the Enterprise CA role

With the `capolicy.inf` in place, I can now install the Active Directory Certificate Services (AD CS) role.

```
Install-WindowsFeature -Name "ADCS-Cert-Authority" -IncludeManagementTools
```

Note! The `-IncludeManagementTools` parameter installs the graphical management tools, such as the Certification Authority MMC. This is useful on servers with a GUI. On Server Core installations it isn't required, as management will be done remotely.

Step 3 – Creating the Enterprise CA certificate request

With the Enterprise CA role installed, the next step is to generate the *certificate request* that will be signed by the Root CA. At this stage, the Enterprise CA does **not** yet have a valid certificate. Instead, it generates a request that defines its identity, cryptographic parameters, and intended role within the PKI hierarchy.

Generating the certificate request

The following command creates the certificate request file:

```
Install-AdcsCertificationAuthority `
    -CACommonName Corp-Enterprise-CA `
```

```
-CAType EnterpriseSubordinateCA `
-CryptoProviderName "RSA#Microsoft Software Key Storage Provider" `
-DatabaseDirectory "C:\Windows\System32\CertLog" `
-HashAlgorithmName SHA256 `
-KeyLength 4096 `
-LogDirectory "C:\Windows\System32\CertLog" `
-OutputCertRequestFile (Join-Path -Path "C:\\" -ChildPath (($env:COMPUTERNAME) +
".req")) `
-Confirm:$false
```

This command prepares the Enterprise CA to become a subordinate of the Root CA by generating a .req file containing:

- The public key
- Cryptographic parameters
- CA identity information

At this stage, the CA is ***not yet active***. It is effectively waiting for approval from the Root CA.

Why there is no certificate lifetime defined here?

You'll notice that no certificate validity period is specified in this step, and that's intentional. For an Enterprise CA, the certificate lifetime is not defined locally. Instead, it is determined by the Root CA at the moment the request is signed.

This is by design:

- The Root CA enforces lifetime policy
- Subordinate CAs cannot extend or override it
- Consistency is maintained across the entire PKI

That's also why the Enterprise CA's capolicy.inf does *not* define validity periods. Any such values would be ignored during issuance. At the end of the command, you'll see a message similar to this:

```
WARNING: The Active Directory Certificate Services installation is incomplete.
To complete the installation, use the request file "C:\LAB-ENT-01.req" to obtain
a certificate from the parent CA. Then, use the Certification Authority snap-in
to install the certificate.
```

```
The operation completed successfully.
0x0 (WIN32: 0)
```

Don't be scared, this is expected. What I need to do now is take the request, transfer it to the root CA, create a certificate, let it be signed by the Root and install it on the Enterprise CA, but let's first complete the Enterprise Server installation first.

Step 4 – Configuring CRL and AIA publication for the Enterprise CA

Now that the Enterprise CA has generated its certificate request, the next step is to configure where it publishes revocation information and how clients discover the issuing CA certificate. Unlike the Root CA, the Enterprise CA is active and continuously issuing certificates. This means its CRL configuration must support:

- Frequent updates
- Delta CRLs
- Reliable client access

Cleaning up default publication points

Just like with the Root CA, Windows creates a set of default CDP and AIA entries that don't always align with a controlled PKI design. To ensure predictable behavior, I first remove all existing entries.

```
$CRLList = Get-CACrldistributionPoint
foreach ( $CRL in $CRLList ) {
    Remove-CACrldistributionPoint $CRL.URI -Confirm:$false | Out-Null
}
```

Configuring CRL publication

The Enterprise CA publishes CRLs more frequently than the Root CA and also supports *Delta CRLs*. This ensures the CA writes both base and delta CRLs locally so they can be distributed externally. Next, I define how clients retrieve this information. This ensures:

- Clients can retrieve full CRLs
- Delta CRLs are supported for faster revocation checking
- URLs remain consistent and web-accessible

Finally, I configure a file-based publication path for operational use. This allows the CA to publish CRLs directly to the SMB share of the web server.

```
# Define CRL file template using ADCS variables
# %3 = CA Name
# %8 = CRL Name Suffix, used when the CA is renewed with a new key
# %9 = Delta CRL indicator, used to distinguish base and delta CRLs

$CRLFileTemplate = "<CAName><CRLNameSuffix><DeltaCRLAllowed>.crl"

$map = @{
    "<CAName>"           = "%3"
    "<CRLNameSuffix>"    = "%8"
    "<DeltaCRLAllowed>" = "%9"
}
```



```

# Replace placeholders with ADCS variables

foreach ($k in $map.Keys) {
    $CRLFileTemplate = $CRLFileTemplate.Replace($k, $map[$k])
}

# Local CRL publication path
# Publishes base and delta CRLs to the local CertEnroll directory

Add-CACrldistributionPoint `
    -Uri "$env:SystemRoot\System32\CertSrv\CertEnroll\$CRLFileTemplate" `
    -PublishToServer `
    -PublishDeltaToServer `
    -Confirm:$false `
    -Force

# HTTP CDP path included in issued certificates
# -AddToCertificateCdp adds this URL to the CDP extension for base CRLs
# -AddToFreshestCrl adds this URL to the Freshest CRL extension for delta CRLs

Add-CACrldistributionPoint `
    -Uri "http://trust.domain.suffix/crl/$CRLFileTemplate" `
    -AddToCertificateCdp `
    -AddToFreshestCrl `
    -Confirm:$false `
    -Force

# UNC publication path used by the CA to publish CRL files
# This path is not included in issued certificates
# Use a real SMB share where the CA computer account has write permissions

Add-CACrldistributionPoint `
    -Uri "file://\trust.domain.suffix\crl\$CRLFileTemplate" `
    -PublishToServer `
    -PublishDeltaToServer `
    -Confirm:$false `
    -Force

```

Note: For the explanation of the ADCS variables (%3, %*, etc) see [“Part 3 – the offline Root CA”](#)

Configuring Authority Information Access (AIA)

Next, I configure where clients can retrieve the issuing CA certificate. First, any default AIA entries are removed:

```

$AIAList = Get-CAAuthorityInformationAccess
foreach ( $AIA in $AIAList ) {
    Remove-CAAuthorityInformationAccess $aia.uri -Confirm:$false
}

```

This ensures that any client validating a certificate can retrieve the issuing CA certificate via HTTP, without needing direct access to the CA itself. Then I define the authoritative AIA location. This ensures that any client validating a certificate can retrieve the issuing CA certificate via HTTP, without needing direct access to the CA itself.

```
# Define AIA file template using ADCS variables
# %3 = CA Name
# %4 = Certificate Name, supports CA certificate renewal with a new key

$AIAFileTemplate = "<CAName><CertificateName>.crt"

$map = @{
    "<CAName>"          = "%3"
    "<CertificateName>" = "%4"
}

# Replace placeholders with ADCS variables
foreach ($k in $map.Keys) {
    $AIAFileTemplate = $AIAFileTemplate.Replace($k, $map[$k])
}

# Local CA certificate publication path
# Publishes the CA certificate to the local CertEnroll directory

Add-CAAuthorityInformationAccess `
    -Uri "$env:SystemRoot\System32\CertSrv\CertEnroll\$AIAFileTemplate" `
    -Confirm:$false `
    -Force

# HTTP AIA path included in issued certificates
# Allows clients to retrieve the issuing CA certificate for chain building

Add-CAAuthorityInformationAccess `
    -Uri "http://trust.domain.suffix/crl/$AIAFileTemplate" `
    -AddToCertificateAia `
    -Confirm:$false `
    -Force
```

Step 5 – Applying registry configuration to the Enterprise CA

Just like with the Root CA, the Enterprise CA relies on several registry-based settings to define how it behaves during normal operation. These settings control CRL timing, overlap windows, and auditing behavior. At this stage, the CA is installed but not yet fully operational. Applying these settings before activating the service ensures consistent and predictable behavior from the start.

```
$Certutil = (Join-Path -Path "$env:SystemRoot" -ChildPath "System32\certutil.exe")
```

```
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLPeriodUnits 1 }  
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLPeriod "Weeks" }  
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLDeltaPeriodUnits 1 }  
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLDeltaPeriod "Days" }  
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLOverlapPeriodUnits 12 }  
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLOverlapPeriod "Hours" }  
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\AuditFilter 127 }
```

At this point the settings will be pretty obvious, but the I'll explain the new ones just in case.

CRL and Delta CRL configuration

The Enterprise CA publishes revocation information more frequently than the Root CA:

- *Base CRL*: every 1 week
- *Delta CRL*: every 1 day

This ensures timely revocation information while keeping network overhead low.

Overlap period

The overlap period ensures that clients never encounter a validation gap when transitioning between CRLs.

By setting a 12-hour overlap, you account for:

- Clock skew
- Cached CRLs
- Temporary connectivity issues

This helps maintain uninterrupted trust validation.

Auditing configuration

Setting AuditFilter to 127 enables full auditing of CA activity, including:

- Certificate issuance
- Revocation events
- Configuration changes

For an Enterprise CA, this level of visibility is essential for troubleshooting, auditing, and compliance.

And that's it for the base configuration of the server, in the next part I'll transfer the request file to the Root CA, Generate the CA certificate, end import it again on the Enterprise server, let's do this!

Step 6 – Signing the Enterprise CA certificate

With the Enterprise CA request generated, the next step is to have it signed by the Root CA. This is the moment where the trust relationship between both CAs is formally established. Until now, the Enterprise CA exists only as a request, it cannot issue certificates yet. Once signed, it becomes a fully trusted subordinate CA.

Step 6.1 – Copy the request to the Root CA

Start by copying the certificate request file from the Enterprise CA to the Root CA. In our example the file is located on the C:\ drive of the Enterprise CA. it looks like C:\<hostname>.req. Copy this file to a temporary working directory on the Root CA. Please note that the request file is just a text file, you can copy the data and past it in another file on the Root CA, if that makes life easier.

Step 6.2 – Sign the Enterprise CA request

On the Root CA, open an *elevated Command Prompt* and navigate to the directory containing the request file. Then run the following command:

```
certreq.exe -config .\<CA NAME> -submit -attrib "CertificateTemplate:SubCA" "<hostname>.req"
```

This submits the request to the Root CA using the SubCA certificate template. After submission, the system will return a *Request ID*. Make a note of this value, it is required for the next step.

Step 6.3 – Issue the certificate

Once the request is submitted, it must be approved and issued:

```
certutil.exe -resubmit <RequestID>
```

This finalizes the certificate issuance process.

Step 6.4 – Retrieve the signed certificate

Now retrieve the signed certificate and store it as a .cer file:

```
certreq.exe -config .\<CA Name> -retrieve <RequestID> "<hostname>.cer"
```

This file contains the signed Enterprise CA certificate.

Step 6.5 – Return the certificate to the Enterprise CA

Copy the newly created <hostname>.cer file back to the Enterprise CA. Once transferred, the Root CA is no longer needed and the signing operation is complete. For security reasons, remove any leftover request or certificate files from the Root CA.

Step 7 – Installing the CA certificates and completing the trust chain

Before the Enterprise CA can be used, a few final certificates must be installed locally. This step completes the trust chain between the Root CA and the Enterprise CA and activates the service. Even though the certificate has been signed, the Enterprise CA does not yet “trust itself” or the Root CA. For the CA service to start correctly, it must:

- Trust the Root CA certificate
- Have its own signed certificate installed
- Be able to validate the full chain locally

Only after this can the Enterprise CA begin issuing certificates.

Step 7.1 – Import the Root CA certificate

Before installing the Enterprise CA certificate, the Root CA certificate must be trusted locally on the Enterprise CA. Reason being that the Enterprise CA needs to trust the Root CA first, this isn't the case just yet, but let's fix that. Run the following command:

```
certutil -addstore -f Root "\\trust.domain.suffix\CRL\Corp-Root-CA.crt"
```

This imports the Root CA certificate into the *Trusted Root Certification Authorities* store. This step ensures that:

- The Enterprise CA recognizes the Root CA as authoritative
- The trust chain can be validated

Step 7.3 – Install the Enterprise CA certificate

Next, install the signed Enterprise CA certificate:

```
certutil.exe -installcert "<hostname>.cer"
```

This binds the signed certificate to the CA service and completes the trust relationship.

Step 7.4 – Start the Certificate Services service

With all certificates in place, start the CA service:

Step 7.5 – Publishing the Enterprise CA certificate to the web server

Once the Enterprise CA is fully installed and running, its public certificate must be made available to clients. This allows systems to build a complete trust chain when validating issued certificates. On the Enterprise CA, navigate to “C:\Windows\System32\CertSrv\CertEnroll”. Copy the .crt file to the CRL directory on the web server. After copying, rename the file so it clearly reflects the issuing CA, like “Corp-Enterprise-CA.cer”.

Note! After starting the Certificate Services (see Step 7.4), CRL files are automatically published to the CRL share. There is no need to manually copy CRL files unless troubleshooting or validating publication.

Step 7.6 – Publishing the Root CA certificate to Active Directory

Required permissions

- Enterprise Adminor
- Delegated rights to publish certificates in Active Directory

Why:

Publishing the Root CA certificate writes to CN=Certification Authorities,CN=Public Key Services,... in Active Directory.

To ensure that all domain-joined systems trust the PKI, the Root CA certificate must be published to Active Directory. Once published, domain members will automatically place the Root CA certificate in their localTrusted Root Certification Authorities store during the next Group Policy refresh cycle. On a system with appropriate permissions (typically the Enterprise CA or a management workstation), open an elevated command prompt and run:

```
certutil -dspublish -f "\\trust.domain.suffix\crl\Corp-Root-CA.crt"
```

The file is published to this LDAP location:

```
ldap:///CN=Corp-Root-CA,CN=Certification Authorities,CN=Public Key  
Services,CN=Services,CN=Configuration,DC=domain,DC=suffix
```

and

```
ldap:///CN=Corp-Root-CA,CN=AIA,CN=Public Key  
Services,CN=Services,CN=Configuration,DC=domain,DC=suffix
```

And I’m done! This completes the entire 4 part setup. Only thing I need to do is validate the installation, let’s do just that.

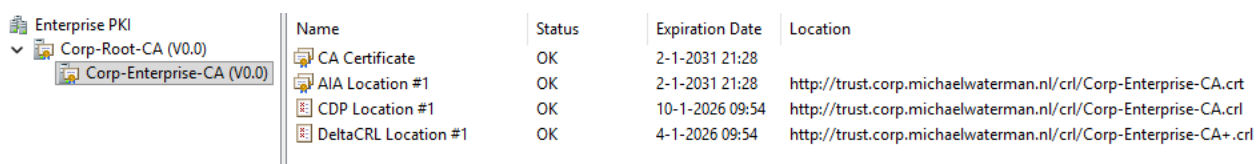
Step 8 – Validating the PKI installation

With both the Root CA and Enterprise CA installed and configured, the final step is to validate that the PKI is functioning as intended. At this point, I want to confirm that:

- The trust chain is intact
- CRLs are reachable
- No configuration errors remain

Start the *PKI Health Tool* by running “pkiview.msc”. This tool provides a consolidated overview of the entire PKI configuration and validates the most critical components. Within PKIView, verify the following:

- The Enterprise CA shows as healthy
- The Root CA is trusted
- CRL and Delta CRL locations are reachable
- No red or critical errors are reported



Name	Status	Expiration Date	Location
CA Certificate	OK	2-1-2031 21:28	
AIA Location #1	OK	2-1-2031 21:28	http://trust.corp.michaelwaterman.nl/crl/Corp-Enterprise-CA.crt
CDP Location #1	OK	10-1-2026 09:54	http://trust.corp.michaelwaterman.nl/crl/Corp-Enterprise-CA.crl
DeltaCRL Location #1	OK	4-1-2026 09:54	http://trust.corp.michaelwaterman.nl/crl/Corp-Enterprise-CA+.crl

Warnings related to offline Root CAs are expected and can safely be ignored, as long as CRL distribution points are accessible. But for now, congratulations. You’ve built a clean, solid, and well-documented PKI!

One last thing, Backing up the Root CA database

After signing the Enterprise CA certificate, the Root CA has completed its primary task. Before taking the Root CA offline again, it is essential to create a full backup of the CA database and private keys. This backup represents the recovery point of your entire PKI trust anchor, even if your Root Server becomes inaccessible, you will still have this backup. The Root CA is rarely used, but it is irreplaceable. If the Root CA is lost without a valid backup:

- The PKI cannot be renewed
- Trust cannot be re-established
- A complete rebuild is the only option

```
Backup-CARoleService `
-Path (Join-Path -Path <BackupPath> -ChildPath ((Get-Date).Ticks)) `
-Password (ConvertTo-SecureString <Password> -AsPlainText -Force)
```

After the backup completes successfully:

- Store the backup offline, redundant and encrypted
- Secure the password separately, for example in a password manager
- Ensure the backup is accessible to more than one trusted administrator

Final thoughts

Over the course of this series, I've built a complete, production-ready on-premises PKI from the ground up. Not by clicking *Next*, *Next*, *Finish*, but by making deliberate design choices, understanding why those choices matter, and automating the process to keep it predictable and repeatable. I started with the fundamentals: architecture, separation of trust, and preparation. From there, I built the distribution layer, established a hardened Offline Root CA, and finally deployed an Enterprise CA that can reliably issue certificates within an Active Directory environment. Along the way, I've focused on:

- Clear trust boundaries
- Explicit configuration over defaults
- Automation using PowerShell (and a bit of command-line)
- Long-term maintainability rather than short-term convenience

The result is a PKI that is not only functional, but understandable, which is arguably the most important part of operating it safely over time. This series intentionally focused on building the PKI. That's only the beginning.

In future posts, I plan to dive deeper into how PKI is actually used in real environments: certificate templates, auto-enrollment, authentication scenarios, operational maintenance, renewals, and the mistakes I still see far too often in production. PKI is one of those technologies that quietly supports everything else. When it's done right, nobody notices. When it's done wrong, everything breaks. My goal with this series was to lower that barrier and show that a well-designed PKI doesn't have to be mysterious or fragile.

If you've followed along this far..... thank you! And if you're just getting started: this foundation will serve you well for years to come.

More to come...

References

[The Microsoft Root Certificate Program](#)

[PKI – Part 1: Introduction to Public Key Infrastructure](#)

[PKI – Part 2: Choosing the key length and algorithm](#)

[PKI – Part 3: The role of hash functions in PKI](#)

[PKI – Part 4: Understanding Cryptographic Providers](#)

[PKI – Part 5: Creating Unique Object Identifiers \(OIDs\)](#)

[PKI – Part 6: Demystifying the CAPolicy.inf file](#)